Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Южно-Уральский государственный университет
(национальный исследовательский университет)»
Институт естественных и точных наук

Отчет
Лабораторная работа №2.
«Броуновское движение частиц»

Выполнил: студент группы ЕТ-310
Тельнов С.М.
Проверил: ассистент кафедры ВМ
Помыкалов Е.В.

Челябинск — 2026

Краткое содержание

- Теоретическая часть 3
- Практическая часть 8
- Вывод 17

Теоретическая часть

В лабораторной работе моделируется плоское движение ансамбля частиц (шаров) в замкнутой прямоугольной области. Частицы движутся равномерно и прямолинейно (сила тяжести отсутствует) и испытывают абсолютно упругие соударения друг с другом и со стенками, благодаря чему суммарная кинетическая энергия системы сохраняется. Визуализация процесса выполняется в реальном времени средствами библиотеки SFML, а управление сведено к минимуму: создание частицы перетаскиванием мыши (вектор натяжения задаёт начальную скорость), изменение радиуса частиц колесом мыши, пауза пробелом и кнопка генерации случайных частиц.

Библиотека SFML

SFML (Simple and Fast Multimedia Library) — это кроссплатформенная объектно-ориентированная библиотека для языка C++, предоставляющая удобный интерфейс к графике, звуку, работе с окнами и сетью. Библиотека разделена на несколько независимых модулей:

- `System` — базовые утилиты: работа со временем (`sf::Clock`, `sf::sleep`), потоками, математические типы `sf::Vector2f`, `sf::Vector2i`;
- `Window` — создание окна `sf::RenderWindow`, обработка событий `sf::Event` (клавиатура, мышь, закрытие окна), ввод;
- `Graphics` — рисование графических примитивов (`sf::CircleShape`, `sf::RectangleShape`, `sf::Text`, `sf::Vertex`), работа с текстурами, шрифтами `sf::Font` и источниками отрисовки;
- `Audio` и `Network` — воспроизведение звука и сетевые соединения (в данной работе не используются).

Библиотека является многопоточной, поддерживает OpenGL и работает на Windows, Linux и macOS. В настоящей работе используются три модуля: `System`, `Window` и `Graphics`.
Проект собирается системой сборки CMake (целевой стандарт — C++17). Поиск библиотеки осуществляется командой `find_package(SFML 3 COMPONENTS Graphics Window System REQUIRED)`, а полученные цели `SFML::Graphics`, `SFML::Window`, `SFML::System` линкуются к исполняемому файлу `program`. Шрифт `DejaVuSans.ttf` копируется в каталог сборки, так как он необходим для вывода текста на экран.
Основная идея архитектуры приложения — классический игровой цикл: на каждой итерации сначала опрашиваются и обрабатываются события окна (`window.pollEvent`), затем выполняется шаг физического моделирования, после чего сцена полностью перерисовывается. Такое разделение удобно, поскольку обработка ввода, расчёт физики и вывод изображения независимы друг от друга.

Модель движения частиц

Каждая частица описывается структурой `Particle`, хранящей координаты центра (x, y), проекции скорости (v_x, v_y) и массу m. Ускорения отсутствуют, поэтому

движение между соударениями — равномерное и прямолинейное. Интегрирование уравнений движения выполняется явным методом Эйлера первого порядка с постоянным шагом Δt:

$$x_i^{n+1} = x_i^n + v_{x,i}^n\,\Delta t, \qquad y_i^{n+1} = y_i^n + v_{y,i}^n\,\Delta t. \tag{1}$$

Шаг интегрирования в программе равен $\Delta t = 0{,}01$ с. Суммарная кинетическая энергия системы вычисляется как

$$E = \sum_{i=1}^{N} \frac{m_i}{2}\left(v_{x,i}^2 + v_{y,i}^2\right), \tag{2}$$

где N — число частиц. Именно эта величина контролируется на экране: для физически корректной модели она должна оставаться постоянной, а отклонение текущей энергии от начальной показывает накопленную численную ошибку.
Отскок от стенок моделируется упруго: если центр частицы вышел за допустимую границу (учитывается радиус r и толщина рамки), координата возвращается на границу, а соответствующая проекция скорости меняет знак, например:

$$x_i < r \;\Rightarrow\; x_i = r, \qquad v_{x,i} = -v_{x,i}. \tag{3}$$

Упругое соударение двух шаров

Наиболее содержательная часть модели — обработка парных соударений шаров. Рассмотрим две частицы 1 и 2 с массами m_1, m_2, положениями $\vec{r}_1$, $\vec{r}_2$ и скоростями $\vec{v}_1$, $\vec{v}_2$. Введём вектор взаимного положения и относительную скорость:

$$\vec{d} = \vec{r}_1 - \vec{r}_2, \qquad d = |\vec{d}|, \qquad \vec{u} = \vec{v}_1 - \vec{v}_2. \tag{4}$$

Соударение возможно, только если расстояние между центрами не превышает суммы радиусов, $d \le 2r$ (в программе сравниваются квадраты расстояний, чтобы избежать извлечения корня). Кроме того, пару обрабатывают лишь тогда, когда частицы сближаются, а не расходятся. Это условие записывается через скалярное произведение

$$\mathrm{dot} = \vec{u}\cdot\vec{d} = (v_{x,1} - v_{x,2})\,d_x + (v_{y,1} - v_{y,2})\,d_y < 0. \tag{5}$$

Если $\mathrm{dot} \ge 0$, частицы уже удаляются друг от друга, и обмена импульсами не происходит.
При абсолютно упругом ударе силы взаимодействия направлены вдоль линии центров, поэтому меняются только нормальные к поверхности касания составляющие скоростей, а тангенциальные сохраняются. Из законов сохранения импульса и кинетической энергии для двух тел получается классическая формула для импульса (импульса удара), передаваемого вдоль нормали $\hat{n} = \vec{d}/d$:

$$J = \frac{2\,m_1 m_2}{m_1 + m_2}\,(\vec{u}\cdot\hat{n}). \tag{6}$$

Новые скорости находятся как

$$\vec{v}_1' = \vec{v}_1 - \frac{J}{m_1}\,\hat{n}, \qquad \vec{v}_2' = \vec{v}_2 + \frac{J}{m_2}\,\hat{n}. \tag{7}$$

Подставляя $\hat{n} = \vec{d}/d$ и $\vec{u} \cdot \hat{n} = \mathrm{dot}/d$, после несложных преобразований получаем формулы, непосредственно реализованные в функции `stepParticles`:

$$\vec{v}_1' = \vec{v}_1 - k_1\,\vec{d}, \qquad \vec{v}_2' = \vec{v}_2 + k_2\,\vec{d}, \tag{8}$$

где коэффициенты

$$k_1 = \frac{2\,m_2}{m_1 + m_2}\,\frac{\mathrm{dot}}{d^2}, \qquad k_2 = \frac{2\,m_1}{m_1 + m_2}\,\frac{\mathrm{dot}}{d^2}. \tag{9}$$

В программе все частицы имеют одинаковую массу $m_1 = m_2 = m$, поэтому формулы упрощаются: $k_1 = k_2 = \mathrm{dot}/d^2$, то есть частицы просто обмениваются нормальными составляющими скоростей.
Из формулы (8) видно, что $m_1\Delta\vec{v}_1 + m_2\Delta\vec{v}_2 = 0$ — закон сохранения импульса выполняется точно. Кинетическая энергия также сохраняется, поскольку удар упругий, что и обеспечивает постоянство величины E из выражения (2).
Отдельно рассмотрим проблему взаимного перекрытия. После одной итерации с конечным шагом Δt частицы могут «застрять» друг в друге. Чтобы этого избежать, после обновления скоростей производится геометрическая коррекция: частицы раздвигаются вдоль нормали на половину глубины проникновения каждая:

$$\vec{r}_1 \mathrel{+}= \frac{d_{\min} - d}{2}\,\hat{n}, \qquad \vec{r}_2 \mathrel{-}= \frac{d_{\min} - d}{2}\,\hat{n}, \qquad d_{\min} = 2r. \tag{10}$$

После сдвига положение каждой частицы ещё раз проверяется на выход за границы области (`resolveWalls`), чтобы коррекция не вытолкнула шар за стенку.

Класс TextBox

Для удобного создания элементов интерфейса (информационных панелей и кнопок управления) реализован вспомогательный класс `TextBox`. Он представляет собой прямоугольный блок с рамкой и центрированным текстом и наследуется от `sf::Drawable`, благодаря чему может отрисовываться тем же вызовом `window.draw`, что и стандартные примитивы SFML. Исходный код класса приведён в листинге 1.
Ключевые решения реализации:

- шрифт `sf::Font` хранится *по значению* внутри класса. Объекты `sf::Text` в SFML хранят лишь указатель на шрифт, поэтому копирование шрифта внутрь виджета гарантирует корректное время жизни и отсутствие висячих ссылок; по этой же причине класс объявлен некопируемым (`delete` для конструктора копирования и оператора присваивания);
- метод `setPosition` не только задаёт положение прямоугольника, но и пересчитывает начало координат текста (`setOrigin`) так, чтобы текст всегда был отцентрирован по блоку. Тот же пересчёт выполняется при любом изменении содержимого или размера шрифта, поэтому виджет не «разъезжается» при смене надписи;
- метод `contains` проверяет попадание курсора в границы блока и используется для обработки нажатий на кнопку (а также для подсветки кнопки при наведении);

- метод `draw` просто рисует прямоугольник и текст, что соответствует контракту интерфейса `sf::Drawable`.

Благодаря такой инкапсуляции создание кнопки сводится к нескольким вызовам настройки (`setSize`, `setPosition`, `setString`, цвета), а вся логика центрирования и отрисовки переиспользуется для всех элементов панели.

```
class TextBox : public sf::Drawable {
    sf::Font font;                      // Шрифт копируется внутрь для
    корректного времени жизни
    sf::Vector2f size;
    sf::Vector2f position;
    sf::RectangleShape box;
    sf::Text text;

public:
    explicit TextBox(const sf::Font& f) : font(f), text(font, "", 14) {
        box.setOutlineThickness(1.f);
    }

    TextBox(const TextBox&) = delete;
    TextBox& operator=(const TextBox&) = delete;

    void setPosition(sf::Vector2f p) {
        position = p;
        box.setPosition(position);

        const sf::FloatRect bounds = text.getLocalBounds();
        text.setOrigin({bounds.position.x + bounds.size.x / 2.f,
                        bounds.position.y + bounds.size.y / 2.f});
        text.setPosition({position.x + size.x / 2.f, position.y + size.y / 2.f
    });
    }

    void setSize(sf::Vector2f s) {
        size = s;
        box.setSize(size);
        setPosition(position);
    }

    void setOutlineBoxThickness(float s) { box.setOutlineThickness(s); }

    void setString(const std::string& s) {
        text.setString(s);
        setPosition(position);
    }

    void setCharacterSize(unsigned s) {
        text.setCharacterSize(s);
        setPosition(position);
    }

    void setTextColor(sf::Color c) {
        text.setFillColor(c);
        text.setOutlineColor(c);
    }

    void setFillBoxColor(sf::Color c) { box.setFillColor(c); }
```

```
    void setOutlineBoxColor(sf::Color c) { box.setOutlineColor(c); }

    /// Проверка попадания точки в границы блока для( кнопок)
    bool contains(sf::Vector2f p) const {
        return p.x >= position.x && p.x <= position.x + size.x &&
               p.y >= position.y && p.y <= position.y + size.y;
    }

    void draw(sf::RenderTarget& target, sf::RenderStates states) const
    override {
        target.draw(box, states);
        target.draw(text, states);
    }

    ~TextBox() override = default;
};
```

Листинг 1. Класс TextBox — вспомогательный UI-виджет

Практическая часть

Ниже приведён полный исходный код программы (`src/main.cpp`). Управление: перетаскивание левой кнопкой мыши задаёт начальную скорость новой частицы, колесо мыши изменяет радиус частиц, пробел ставит симуляцию на паузу, кнопка «+10 random balls» добавляет десять случайных частиц. Параметры каждой итерации (номер, число частиц, радиус, кинетическая энергия и её отклонение от начальной) записываются в файл `simulation_log.csv`.

```
// Моделирование 2Dдвижения- и упругих соударений частиц без( гравитации).
//
// Управление:
//   ЛКМ + drag & drop   – создание частицы вектор( натяжения задает скорость)
    ;
//   Колесо мыши         – изменение радиуса частиц;
//   Пробел              – пауза;
//   Кнопка «+10 random» – генерация 10 случайных частиц.
//
// Логирование: параметры каждой итерации пишутся в simulation_log.csv.

#include <SFML/Graphics.hpp>
#include <algorithm>
#include <cmath>
#include <cstdio>
#include <fstream>
#include <iostream>
#include <optional>
#include <random>
#include <string>
#include <vector>

/// Физическое состояние частицы
struct Particle {
    float x;                                // Xкоордината- центра м()
    float y;                                // Yкоордината- центра м()
    float vx;                               // Скорость по X мс(/)
    float vy;                               // Скорость по Y мс(/)
    float m;                                // Масса кг()
    sf::Color color = sf::Color::Green;
};

/**
 * @brief Численный шаг интегрирования: метод Эйлера, упругие удары о стенки
    и соударения шаров.
 * @param particles Список частиц симуляции.
 * @param dt Временной шаг с().
 * @param radius Радиус шара м().
 * @param boxW Ширина рабочей области м().
 * @param boxH Высота рабочей области м().
 * @param border Толщина внешней рамки м().
 */
void stepParticles(std::vector<Particle>& particles, float dt, float radius,
                   float boxW, float boxH, float border)
{
    const float minX = border + radius;
    const float maxX = boxW - border - radius;
    const float minY = border + radius;
```

```
    const float maxY = boxH - border - radius;

    // Упругий отскок от границ
    auto resolveWalls = [&](Particle& p) {
        if (p.x < minX) { p.x = minX; if (p.vx < 0.f) p.vx = -p.vx; }
        if (p.x > maxX) { p.x = maxX; if (p.vx > 0.f) p.vx = -p.vx; }
        if (p.y < minY) { p.y = minY; if (p.vy < 0.f) p.vy = -p.vy; }
        if (p.y > maxY) { p.y = maxY; if (p.vy > 0.f) p.vy = -p.vy; }
    };

    // 1. Интегрирование координат и обработка стенок
    for (auto& p : particles) {
        p.x += p.vx * dt;
        p.y += p.vy * dt;
        resolveWalls(p);
    }

    // 2. Парные упругие соударения шаров
    const float minDist = 2.f * radius;
    const float minDistSq = minDist * minDist;

    for (size_t i = 0; i < particles.size(); ++i) {
        for (size_t j = i + 1; j < particles.size(); ++j) {
            Particle& p1 = particles[i];
            Particle& p2 = particles[j];

            const float dx = p1.x - p2.x;
            const float dy = p1.y - p2.y;
            const float distSq = dx * dx + dy * dy;

            if (distSq <= 0.f || distSq > minDistSq)
                continue;

            const float dvx = p1.vx - p2.vx;
            const float dvy = p1.vy - p2.vy;
            const float dot = dvx * dx + dvy * dy;

            // Обрабатываем только сближающиеся частицы
            if (dot >= 0.f)
                continue;

            const float dist = std::sqrt(distSq);
            const float totalMass = p1.m + p2.m;
            const float k1 = (2.0f * p2.m) / totalMass * (dot / distSq);
            const float k2 = (2.0f * p1.m) / totalMass * (dot / distSq);

            // Обновление скоростей
            p1.vx -= k1 * dx;
            p1.vy -= k1 * dy;
            p2.vx += k2 * dx;
            p2.vy += k2 * dy;

            // Коррекция взаимного перекрытия
            const float overlap = 0.5f * (minDist - dist);
            const float nx = dx / dist;
            const float ny = dy / dist;
            p1.x += nx * overlap;
            p1.y += ny * overlap;
```

```
            p2.x -= nx * overlap;
            p2.y -= ny * overlap;

            resolveWalls(p1);
            resolveWalls(p2);
        }
    }
}

/// Вычисление суммарной кинетической энергии системы Дж()
float kineticEnergy(const std::vector<Particle>& particles)
{
    float energy = 0.f;
    for (const Particle& p : particles)
        energy += (p.vx * p.vx + p.vy * p.vy) * p.m / 2.f;
    return energy;
}

/// Форматирование числа с фиксированным числом знаков после запятой
std::string formatNumber(float value, int decimals)
{
    char buffer[32];
    std::snprintf(buffer, sizeof(buffer), "%.*f", decimals, value);
    return std::string(buffer);
}

/// UIвиджет- блока с центрированным текстом и границей
class TextBox : public sf::Drawable {
    sf::Font font;                      // Шрифт копируется внутрь для
   корректного времени жизни
    sf::Vector2f size;
    sf::Vector2f position;
    sf::RectangleShape box;
    sf::Text text;

public:
    explicit TextBox(const sf::Font& f) : font(f), text(font, "", 14) {
        box.setOutlineThickness(1.f);
    }

    TextBox(const TextBox&) = delete;
    TextBox& operator=(const TextBox&) = delete;

    void setPosition(sf::Vector2f p) {
        position = p;
        box.setPosition(position);

        const sf::FloatRect bounds = text.getLocalBounds();
        text.setOrigin({bounds.position.x + bounds.size.x / 2.f,
                        bounds.position.y + bounds.size.y / 2.f});
        text.setPosition({position.x + size.x / 2.f, position.y + size.y / 2.f
   });
    }

    void setSize(sf::Vector2f s) {
        size = s;
        box.setSize(size);
        setPosition(position);
```

```
    }

    void setOutlineBoxThickness(float s) { box.setOutlineThickness(s); }

    void setString(const std::string& s) {
        text.setString(s);
        setPosition(position);
    }

    void setCharacterSize(unsigned s) {
        text.setCharacterSize(s);
        setPosition(position);
    }

    void setTextColor(sf::Color c) {
        text.setFillColor(c);
        text.setOutlineColor(c);
    }

    void setFillBoxColor(sf::Color c) { box.setFillColor(c); }
    void setOutlineBoxColor(sf::Color c) { box.setOutlineColor(c); }

    /// Проверка попадания точки в границы блока для( кнопок)
    bool contains(sf::Vector2f p) const {
        return p.x >= position.x && p.x <= position.x + size.x &&
               p.y >= position.y && p.y <= position.y + size.y;
    }

    void draw(sf::RenderTarget& target, sf::RenderStates states) const
    override {
        target.draw(box, states);
        target.draw(text, states);
    }

    ~TextBox() override = default;
};

int main()
{
    const float dt = 0.01f;
    const unsigned winW = 800;
    const unsigned winH = 600;
    const float boxW = static_cast<float>(winW);
    const float boxH = static_cast<float>(winH);
    const float border = 1.f;

    sf::RenderWindow window(sf::VideoMode({winW, winH}), "LR2: particle flight
    ");

    // Рамка границ области симуляции
    sf::Vertex boxBorders[] = {
        {{0.f, winH - 1.f}, sf::Color::White}, {{boxW, winH - 1.f}, sf::Color
    ::White},
        {{0.f, 1.f}, sf::Color::White},        {{boxW, 1.f}, sf::Color::White
    },
        {{1.f, 0.f}, sf::Color::White},        {{1.f, boxH}, sf::Color::White
    },
        {{winW - 1.f, 0.f}, sf::Color::White}, {{winW - 1.f, boxH}, sf::Color
```

```
    ::White}
    };

    float radius = 5.f;
    const float mass = 1.0f;
    bool isPaused = false;
    std::vector<Particle> particles;

    const sf::Color colors[] = {
        sf::Color::Green, sf::Color::Yellow, sf::Color::Cyan,
        sf::Color::Magenta, sf::Color::Red
    };
    size_t colorIdx = 0;

    sf::CircleShape circle(radius);

    // Ограничение координат центра шара рамкой окна
    const auto clampInside = [&](const sf::Vector2f& p) {
        return sf::Vector2f(std::clamp(p.x, border + radius, boxW - border -
    radius),
                            std::clamp(p.y, border + radius, boxH - border -
    radius));
    };

    // Состояние прицеливания задание( начальной скорости)
    bool isAiming = false;
    sf::Vector2f aimStartPx;
    sf::Vector2f aimCurrentPx;

    float currentEnergy = 0.f;
    float initialEnergy = 0.f;

    sf::Font font;
    if (!font.openFromFile("./DejaVuSans.ttf")) {
        return -1;
    }

    // Инициализация виджетов панели метрик
    const sf::Color buttonIdle(73, 79, 82);
    const sf::Color buttonHover(34, 102, 54);

    auto setupWidget = [&](TextBox& w, float xPos) {
        w.setSize({126.f, 36.f});
        w.setPosition({xPos, 6.f});
        w.setOutlineBoxThickness(-1.f);
        w.setCharacterSize(11);
        w.setFillBoxColor(buttonIdle);
        w.setOutlineBoxColor(sf::Color::White);
        w.setTextColor(sf::Color::White);
    };

    TextBox energyText(font);     setupWidget(energyText, 6.f);
    TextBox errorText(font);      setupWidget(errorText, 136.f);
    TextBox iterationsText(font); setupWidget(iterationsText, 266.f);
    TextBox radiusText(font);     setupWidget(radiusText, 396.f);
    TextBox ballsText(font);      setupWidget(ballsText, 526.f);
    TextBox addBallsButton(font); setupWidget(addBallsButton, 656.f);

```

```
    iterationsText.setString("Iterations:\n0");
    radiusText.setString("Radius:\n" + formatNumber(radius, 1) + " px");
    ballsText.setString("Balls:\n0");
    addBallsButton.setString("+10 random balls");

    // Инициализация CSVжурнала-
    std::ofstream logFile("simulation_log.csv");
    if (!logFile)
        std::cerr << "Не удалось открыть simulation_log.csv для записи\n";
    logFile << "iteration,balls,radius,kinetic_energy,error\n";

    unsigned long long iteration = 0;

    // Выполнение шага моделирования, обновление энергии и лога
    const auto runIteration = [&]() {
        stepParticles(particles, dt, radius, boxW, boxH, border);
        ++iteration;
        currentEnergy = kineticEnergy(particles);

        logFile << iteration << ',' << particles.size() << ',' << radius << ',
'
                << currentEnergy << ',' << (currentEnergy - initialEnergy) <<
'\n';

        if (iteration % 100 == 0) {
            iterationsText.setString("Iterations:\n" + std::to_string(
iteration));
            logFile.flush();
        }
    };

    // Спавн n непересекающихся частиц со случайной скоростью
    std::mt19937 rng(std::random_device{}());
    const auto addRandomParticles = [&](int count) {
        std::uniform_real_distribution<float> posX(border + radius, boxW -
border - radius);
        std::uniform_real_distribution<float> posY(border + radius, boxH -
border - radius);
        std::uniform_real_distribution<float> angle(0.f, 2.f * std::acos(-1.0f
));
        std::uniform_real_distribution<float> speed(25.f, 600.f);

        const float energyBefore = kineticEnergy(particles);
        for (int i = 0; i < count; ++i) {
            sf::Vector2f pos(posX(rng), posY(rng));
            for (int attempt = 0; attempt < 100; ++attempt) {
                bool overlaps = false;
                for (const Particle& p : particles) {
                    const float dx = pos.x - p.x;
                    const float dy = pos.y - p.y;
                    if (dx * dx + dy * dy < 4.f * radius * radius) {
                        overlaps = true;
                        break;
                    }
                }
                if (!overlaps) break;
                pos = sf::Vector2f(posX(rng), posY(rng));
            }
```

```
321
322             const float a = angle(rng);
323             const float v = speed(rng);
324             particles.push_back({pos.x, pos.y, v * std::cos(a), v * std::sin(a
    ),
325                                  mass, colors[colorIdx++ % 5]});
326         }
327
328         initialEnergy += kineticEnergy(particles) - energyBefore;
329         currentEnergy = kineticEnergy(particles);
330     };
331
332     // Основной цикл приложения
333     while (window.isOpen()) {
334         while (const std::optional event = window.pollEvent()) {
335             if (event->is<sf::Event::Closed>()) {
336                 window.close();
337             } else if (const auto* key = event->getIf<sf::Event::KeyPressed>()
    ) {
338                 if (key->code == sf::Keyboard::Key::Space)
339                     isPaused = !isPaused;
340             } else if (const auto* wheel = event->getIf<sf::Event::
    MouseWheelScrolled>()) {
341                 if (wheel->wheel == sf::Mouse::Wheel::Vertical)
342                     radius = std::clamp(radius + wheel->delta, 2.0f, 50.0f);
343             } else if (const auto* mouse = event->getIf<sf::Event::
    MouseButtonPressed>()) {
344                 if (mouse->button == sf::Mouse::Button::Left) {
345                     const sf::Vector2f clickPos(static_cast<float>(mouse->
    position.x),
346                                                 static_cast<float>(mouse->
    position.y));
347                     if (addBallsButton.contains(clickPos)) {
348                         addRandomParticles(10);
349                     } else {
350                         isAiming = true;
351                         aimStartPx = clickPos;
352                         aimCurrentPx = aimStartPx;
353                     }
354                 }
355             } else if (const auto* mouseMove = event->getIf<sf::Event::
    MouseMoved>()) {
356                 const sf::Vector2f mousePos(static_cast<float>(mouseMove->
    position.x),
357                                             static_cast<float>(mouseMove->
    position.y));
358                 addBallsButton.setFillBoxColor(addBallsButton.contains(
    mousePos) ? buttonHover : buttonIdle);
359                 if (isAiming) aimCurrentPx = mousePos;
360             } else if (const auto* mouseRelease = event->getIf<sf::Event::
    MouseButtonReleased>()) {
361                 if (mouseRelease->button == sf::Mouse::Button::Left &&
    isAiming) {
362                     isAiming = false;
363                     aimCurrentPx = sf::Vector2f(static_cast<float>(
    mouseRelease->position.x),
364                                                 static_cast<float>(
    mouseRelease->position.y));
```

```
                const sf::Vector2f spawn = clampInside(aimStartPx);
                const float vx = aimStartPx.x - aimCurrentPx.x;
                const float vy = aimStartPx.y - aimCurrentPx.y;

                particles.push_back({spawn.x, spawn.y, vx, vy, mass,
colors[colorIdx++ % 5]});
                initialEnergy += (vx * vx + vy * vy) * mass / 2.f;
                currentEnergy = kineticEnergy(particles);
            }
        }
    }

    if (!isPaused) {
        runIteration();
    }

    // Рендеринг
    window.clear(sf::Color::Black);
    window.draw(boxBorders, 8, sf::PrimitiveType::Lines);

    circle.setRadius(radius);
    circle.setOrigin({radius, radius});

    for (const auto& p : particles) {
        circle.setFillColor(p.color);
        circle.setPosition({p.x, p.y});
        window.draw(circle);
    }

    // Вектор натяжения и превью создаваемого шара
    if (isAiming) {
        sf::Vertex aimLine[] = {
            {aimStartPx, sf::Color::Yellow},
            {aimCurrentPx, sf::Color::Red}
        };
        window.draw(aimLine, 2, sf::PrimitiveType::Lines);

        circle.setFillColor(sf::Color(255, 255, 255, 128));
        circle.setPosition(clampInside(aimStartPx));
        window.draw(circle);
    }

    // Отрисовка UI
    energyText.setString("Kinetic energy:\n" + formatNumber(currentEnergy,
 2) + " J");
    errorText.setString("Error:\n" + formatNumber(currentEnergy -
initialEnergy, 2) + " J");
    radiusText.setString("Radius:\n" + formatNumber(radius, 1) + " px");
    ballsText.setString("Balls:\n" + std::to_string(particles.size()));

    window.draw(energyText);
    window.draw(errorText);
    window.draw(radiusText);
    window.draw(ballsText);
    window.draw(iterationsText);
    window.draw(addBallsButton);

```

```
        window.display();
        sf::sleep(isPaused ? sf::milliseconds(16) : sf::seconds(dt));
    }

    logFile.flush();
    logFile.close();
    return 0;
}
```

Листинг 2. Полный исходный код программы (src/main.cpp)

Вывод

В ходе лабораторной работы была разработана программа, моделирующая физически правдоподобное броуновское движение ансамбля частиц в замкнутой области. Движение реализовано методом Эйлера, а взаимодействие шаров друг с другом и со стенками — абсолютно упругими соударениями, что обеспечивает сохранение суммарной кинетической энергии системы.
Для визуализации процесса использована библиотека SFML, организованная по принципу игрового цикла «события — физика — отрисовка». Для удобного построения интерфейса реализован класс `TextBox`, инкапсулирующий прямоугольный блок с центрированным текстом и поддерживающий обработку нажатий. Управление программой сведено к минимуму кнопок и жестов мыши. Результаты работы программы контролируются выводом на экран текущей кинетической энергии и её отклонения от начальной: наблюдаемое отклонение остаётся малым, что подтверждает корректность выбранных формул упругого удара и сохранение энергии в модели.